

Pilhas e Filas

Conjuntos dinâmicos com política de remoção definida.

CLRS – Algoritmos: Teoria e Prática, Capítulo 10.1

1-4 Pilha

Conceito LIFO

Manual e STL

Exercício

5-7 Fila

Conceito FIFO

Manual e STL

Exercício

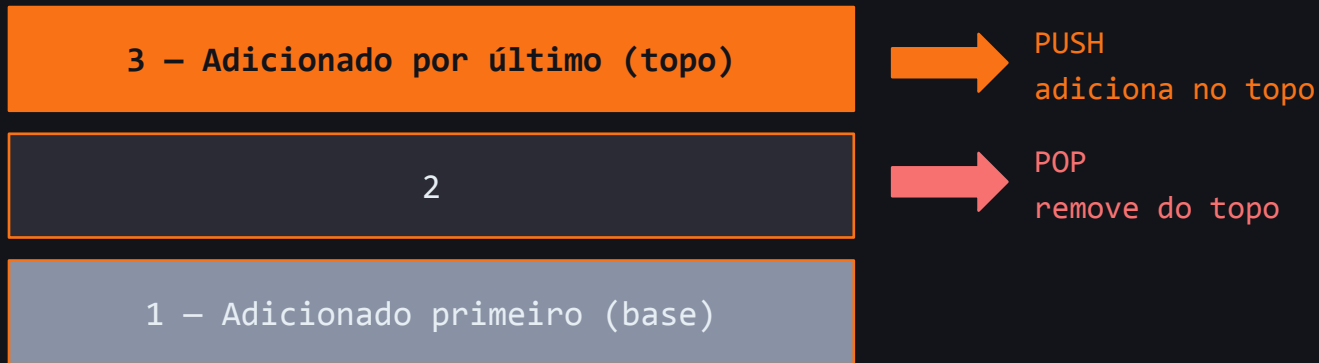
8 Deque

Pilha + Fila
una só biblioteca

Pilha – LIFO (Last In, First Out)

CLRS: "Em uma pilha, o elemento eliminado do conjunto é o mais recentemente inserido – a pilha implementa uma política de último a entrar, primeiro a sair."

Analogia: pilha de pratos em um restaurante.



PUSH(S, x)

→ Insere x no topo da pilha.

POP(S)

→ Remove e retorna o elemento do topo.

EMPTY(S)

→ Retorna verdadeiro se a pilha estiver vazia.

Pilha – implementação com array

CLRS: "Podemos implementar uma pilha de no máximo n elementos com um arranjo $S[1..n]$. O arranjo tem um atributo $S.topo$ que indexa o elemento mais recentemente inserido."

```
#include<iostream>
using namespace std;
const int MAX = 100;
int S[MAX]; // o array que armazena os elementos
int topo = 0; // S.topo – índice do elemento no topo
bool EMPTY() { return topo == 0; }
void PUSH(int x) {
    if (topo >= MAX) { cout << "Estouro!"; return; }
    S[++topo] = x; // incrementa topo e insere
}
int POP() {
    if (EMPTY()) { cout << "Pilha vazia!"; return -1; }
    return S[topo--]; // retorna e decrementa topo
}
```

Pilha – usando #include<stack>

A STL do C++ já tem uma implementação pronta. Na prática, usamos ela – a manual serve para entender o que acontece por dentro.

```
#include<iostream>
#include<stack>
using namespace std;

int main() {
    stack<int> S;

    S.push(4);
    S.push(1);
    S.push(3);
    cout << S.top(); // 3
    S.pop();
    cout << S.top(); // 1
```

Funções disponíveis:

push(x)

insere x no topo

pop()

remove o topo

top()

retorna o topo sem remover

empty()

true se vazia

size()

número de elementos

Exercício – verificador de parênteses

Escreva um programa que receba uma string e verifique se os parênteses, colchetes e chaves estão corretamente balanceados.

Como a pilha processa {[()]}



📌 Exercício:

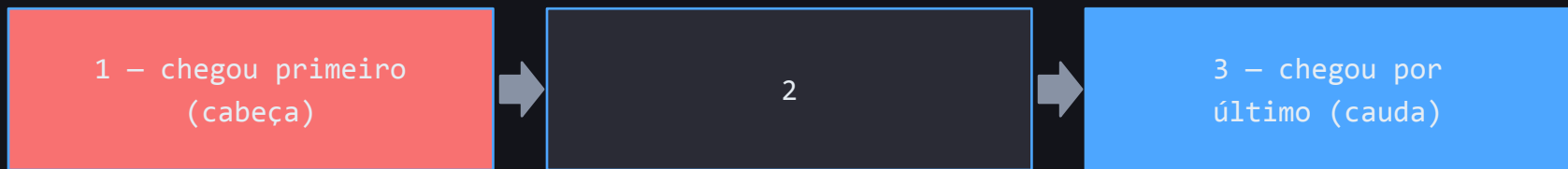
Implemente em C++ usando `stack<char>`. Teste com: `{[()]}` → inválido | `([])` → válido
| `{[]()}` → válido

Fila – FIFO (First In, First Out)

CLRS: "Em uma fila, o elemento eliminado é sempre o que estava no conjunto há mais tempo – a fila implementa uma política de primeiro a entrar, primeiro a sair."

Analogia: fila de atendimento em um banco.

← DEQUEUE (sai pela frente)



ENQUEUE → (entra pelo fim)

ENQUEUE(Q, x)

→ Insere x no fim da fila.

DEQUEUE(Q)

→ Remove e retorna o elemento da frente (cabeça).

EMPTY(Q)

→ Retorna verdadeiro se $Q.início == Q.fim$.

Fila – usando #include<queue>

A STL do C++ já tem uma implementação pronta. Mesma lógica do stack – usamos na prática, a manual serve para entender o que acontece por dentro.

```
#include<iostream>
#include<queue>
using namespace std;

int main() {
    queue<string> fila;

    // ENQUEUE – insere no fim
    fila.push("Ana");
    fila.push("Bruno");
    fila.push("Carla");

    // DEQUEUE – remove da frente
    cout << fila.front(); // Ana
    fila.pop();
```

Funções:

push(x)
insere x no fim

pop()
remove da frente

front()
lê a frente sem remover

back()
lê o fim sem remover

empty()
true se vazia

size()
número de elementos

Exercício – sistema de gerenciamento de fila

Sistema de atendimento com senhas numéricas – o cliente retira um bilhete e aguarda o atendente chamar.

retirar_bilhete()

Gera o próximo número e enfileira o cliente.

proximo()

Chama o primeiro da fila e o remove. Exibe o número chamado.

exibir_fila()

Percorre toda a fila do primeiro ao último exibindo os números.

vazia()

Retorna true se não há ninguém aguardando.

 **Exercício:**

Simule: 5 clientes retiram bilhetes → exibir_fila() → proximo() 3x → 2 novos clientes → exibir_fila() → proximo() até vazia().

Deque – double-ended queue

CLRS 10.1-5: uma deque permite inserção e remoção em ambas as extremidades – combina o comportamento de pilha e fila na mesma estrutura.



Como pilha (LIFO): push_back() empilha no fim + pop_back() desempilha do fim – mesmo lado.

Como fila (FIFO): push_back() enfileira no fim + pop_front() desenfileira da frente – lados opostos.

Deque puro: Inserção e remoção em qualquer extremidade – mais flexível que pilha ou fila isoladas.

Deque – usando #include<deque>

A biblioteca padrão do C++ oferece a deque pronta. Ela tem todas as funções de stack e queue – e mais para as duas extremidades.

```
#include<iostream>
#include<deque>
using namespace std;
int main() {
    deque<int> dq;
    // Usando como FILA – push_back/pop_front
    dq.push_back(1);    // [1]
    dq.push_back(2);    // [1, 2]
    dq.push_back(3);    // [1, 2, 3]
    cout << dq.front(); // 1 – FIFO
    dq.pop_front();     // [2, 3]
    // Usando como PILHA – push_back/pop_back
    dq.push_back(4);     // [2, 3, 4]
    cout << dq.back();   // 4 – LIFO
    dq.pop_back();       // [2, 3]
```

Funções:

push_front(x)
insere na frente

push_back(x)
insere no fim

pop_front()
remove da frente

pop_back()
remove do fim

front()
lê a frente

back()
lê o fim

empty()
true se vazia

Exercício – atendimento prioritário

O sistema que você implementou atende todos os clientes na mesma ordem. Surge um novo requisito:

A clínica agora atende idosos, gestantes e pessoas com deficiência com prioridade – antes dos demais, mesmo que tenham chegado depois.

Por que a fila simples não resolve?

- 1 A queue só permite inserção no fim e remoção na frente – não há como inserir alguém à frente dos outros.
- 2 Remover da fila e reordenar seria caro e quebraria a lógica FIFO para os clientes comuns.
- 3 Precisamos de uma estrutura que permita inserção em ambas as extremidades.

Solução: Use a deque (`#include<deque>`) – ela permite inserção e remoção em ambas as extremidades, combinando fila e pilha em uma só estrutura.